

Gut Microbiota and Sepsis-Associated Acute Kidney Injury

Katherine Winner

2024-05-29

Contents

I. Introduction	1
A. Prepare environment	1
B. Color Palette Prep	5
II. Bacterial Density	5
A. Sequencing Controls versus Swab Specimens	5
B. Cases (developed AKI) versus Controls (did not develop AKI)	6
III. Community Composition	8
A. Principal Component Analysis	8
B. Hypothesis Testing	12
C. Random Forest	13
IV. Community Diversity	16
A. Beta-diversity: across specimens	16
B. Alpha-diversity: within specimens	18

This report was built with R version 4.4.0

I. Introduction

In this document we analyze swabs from patients admitted to the University of Michigan hospital with suspected sepsis. We have matched patients who developed AKI with controls who did not by sex, age, and unit of admission. Here we look at the identity and density of gut microbiota via 16S rRNA amplicon sequencing and ddPCR quantification of these rectal swab specimens to determine if the gut microbiome is predictive of AKI.

A. Prepare environment

1. Library

```

# universal data wrangling and visualization
library(tidyverse) # data manipulation with dplyr
library(ggplot2)
library(ggpubr) # to add statistical testing to plots
library(broom) # convert statistical model output to data frame format
library(ggbeeswarm) # to plot beeswarm plots and quasirandom plots
library(readxl)

# machine learning and analysis packages
library(vegan) # community ecology analysis
library(ranger)
library(fossil)
library(mvabund)
library(randomForest)
library(caret) # machine learning tuning package
library(DMwR2) # for SMOTE - Synthetic Minority Oversampling Technique
library(survival) # for survival analysis

# formatting, plotting
library(scales)
library(RColorBrewer)
library(knitr) # import csv files
library(tidytext) # text wrangling with tidyverse syntax
library(kableExtra) # to display tables in Rmarkdown
library(extrafont) # to adjust font type
loadfonts(device = "win")

# create a function for standard error
sem <- function(x){
  sqrt(var(x, na.rm = TRUE)/length(na.omit(x)))
}

```

2. 16S Data

Load in the 16S data from two mothur files which have the raw counts and taxonomy information

```

# For ease of use, place the 16S data files of interest into the same
#directory as current RStudio project

# read in .shared (counts of sequences assigned to OTU #s)
# basic processing and selection of OTUs
otu_raw <- read.table("aki_mothur.opti_mcc.shared", row.names = 2,
                     header = T, stringsAsFactors = F)
otu_trim <- otu_raw[, -c(1:2)] # drop the first 2 columns
otu_trim <- as.matrix(otu_trim)
otu_tmp <- vegan::decostand(otu_trim, "total") * 100 # standardization
otu_trim[which(otu_tmp < 0.1)] <- 0 # select OTU > 0.1% of population
otu_good <- otu_trim[, which(colSums(otu_trim) > 0)] # keep only the OTUs
# which were present after setting quantities under 0.1 to 0
# edit rownames
# function to move "RX_" from the beginning to the end after
# removing "_L001_R1_001" from the end of row names

```

```

modify_row_names <- function(name) {
  # remove "_L001_Rx_001" from the end
  name <- sub("_L001_R1_001$", "", name)
  # extract "RX" from the beginning
  prefix <- sub("^(R[1-3]).*", "\\1", name)
  # remove "RX_" from the beginning
  name <- sub("^R[1-3]_", "", name)
  # append the prefix to the end
  name <- paste0(name, "_", prefix)
  # remove "Woods" from the beginning
  name <- sub("^Woods_", "", name)
  return(name)
}
# apply the function to the row names
rownames(otu_good) <- sapply(rownames(otu_good), modify_row_names)

# read in cons.taxonomy (for each OTU: contains taxonomic identifiers
# down to Genus level, if possible)
# two imports are required due to the combination of 2 different separators
tax1 <- read.table("aki_mothur.opti_mcc.0.03.cons.taxonomy",
  sep = "\t", row.names = 1,
  header = TRUE,
  colClasses = c("character", "numeric", "character"))
tax2 <- read.table("aki_mothur.opti_mcc.0.03.cons.taxonomy", sep = ";",
  skip = 1,
  col.names = c("", "Phylum", "Class", "Order", "Family", "Genus", ""),
  stringsAsFactors = FALSE) %>%
  dplyr::select(-c(1, 7)) %>%
  purrr::map_df(stringr::str_replace, "\\(.*\\.\\)", "") %>%
  purrr::map_df(as.factor) # turns all present columns into factor type variables
otu_taxonomy <- data.frame(OTU = rownames(tax1),
  Size = tax1[, 1], tax2,
  stringsAsFactors = FALSE) # combine tax1 and tax2
rownames(otu_taxonomy) <- rownames(tax1)

# trim otu_taxonomy based on OTUs in otu_good
otu_good_taxonomy <- otu_taxonomy[otu_taxonomy$OTU %in% colnames(otu_good), ] %>%
  droplevels() # remove factor levels that have no values due to removal
rownames(otu_good_taxonomy) <- intersect(rownames(otu_taxonomy), colnames(otu_good))

```

3. Metadata

Create sample name, sample type (swab or sequencing control), and sequencing run from otu_good rownames. Add AKI definition, match ID, and ddPCR reads per sample

```

# have to create own metadata table
otu_df <- data.frame(decostand(otu_good, "total") * 100, sample_name = row.names(otu_good),
  stringsAsFactors = FALSE) %>%
  # use sample name to label sequencing controls and rectal swab specimens
  mutate(sample = factor(case_when(str_detect(sample_name, "^AE_") ~ "AE",
    str_detect(sample_name, "^Blank_") ~ "Empty",
    str_detect(sample_name, "^BLANK_") ~ "Empty",
    str_detect(sample_name, "^IsoCtrl_") ~ "IsoCtrl",

```

```

      str_detect(sample_name, "^Empty_") ~ "Empty",
      str_detect(sample_name, "^Water_NEG") ~ "WaterNeg",
      str_detect(sample_name, "^WaterNegCtrl") ~ "WaterNeg",
      str_detect(sample_name, "^Zymo_POS") ~ "Mock",
      str_detect(sample_name, "^Zymo") ~ "Mock",
      str_detect(sample_name, "^PR") ~ "Swab"),
    levels = c("Swab", "AE", "Empty", "WaterNeg", "IsoCtrl", "Mock")),
  # 2 runs were used, make a column to determine which run from sample_name
  run = factor(case_when(str_ends(sample_name, "_R1") ~ 1,
    str_ends(sample_name, "_R2") ~ 2),
    levels = c("1", "2"))) %>%
dplyr::select(sample_name, sample, run, everything())

# read in aki definition, match ID, and ddPCR_reads_per_sample
metadata <- read_csv("processed_metadata.csv")
# add metadata to otu_df
all_data <- inner_join(metadata, otu_df, by = "sample_name")

```

```

# get metadata for swab specimens only
# make all character columns factors
all_data_swabs <- all_data %>%
  filter(!is.na(aki)) %>%
  mutate(sample_name = factor(sample_name),
    match_id = factor(match_id),
    aki = factor(aki))
all_data_swabs <- as.data.frame(all_data_swabs) # change from tibble to data frame
# for adding row names
rownames(all_data_swabs) <- all_data_swabs$sample_name # add row names
# for PCA analysis later
# note that data is in wide format

```

a. Just swab data

```

# gather the OTU from wide to long
swab_data_tidy <- all_data_swabs %>%
  gather("Otu", "relative_abundance", -c(sample_name:run)) %>%
  mutate(Otu = as.factor(Otu))

# link OTU to genus
otu_genus <- otu_good_taxonomy %>%
  dplyr::select(OTU, Genus, Phylum) %>%
  mutate(Otu_genus = str_c(OTU, Genus, sep = "-")) %>%
  rename("Otu" = OTU) %>%
  mutate(Otu = as_factor(Otu))
tidy_genus_df <- inner_join(swab_data_tidy, otu_genus, by = "Otu") %>%
  mutate(relative_abundance = as.numeric(relative_abundance))

```

```
# family-level abundance data frame
fam_df <- all_data_swabs %>%
  dplyr::select(sample_name, contains("Otu")) %>%
  tidyr::pivot_longer(cols = -stringr::str_which(colnames(.), "Otu", negate = TRUE),
    names_to = "OTU", values_to = "Percentage") %>%
  dplyr::inner_join(dplyr::select(otu_good_taxonomy, OTU, Family), by = "OTU") %>%
  dplyr::group_by(sample_name, Family) %>%
  summarize(Percentage = sum(Percentage), .groups = "drop") %>%
  tidyr::spread(key = Family, value = Percentage) %>%
  dplyr::inner_join(dplyr::select(all_data_swabs, -contains("Otu")),
    by = "sample_name") %>%
  dplyr::select(sample_name, aki:run, everything())
```

b. Tidy swab data

B. Color Palette Prep

Create a custom color palette for visualizations

```
pal <- c("#E69F00", "#56B4E9", "#009E73", "#00e6ab", "#F0E442",
  "#0072B2", "#D55E00", "#CC79A7", "#e1b0cb", "#999999")
```

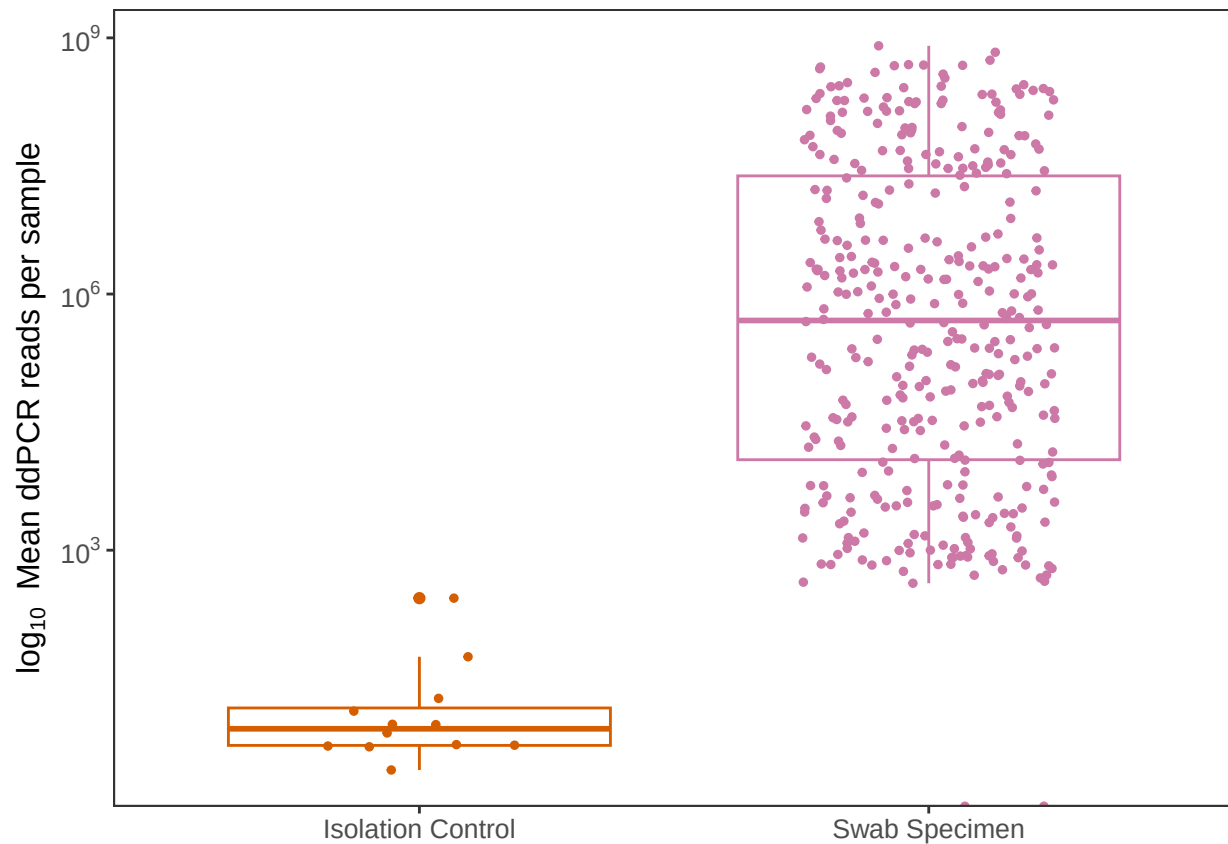
II. Bacterial Density

A. Sequencing Controls versus Swab Specimens

```
# bacterial density table by sample type
all_data %>%
  group_by(sample) %>%
  summarize(`Mean` = mean(ddPCR_reads_per_sample, na.rm = T),
    `Median` = median(ddPCR_reads_per_sample, na.rm = T),
    `Minimum` = min(ddPCR_reads_per_sample, na.rm = T),
    `Maximum` = max(ddPCR_reads_per_sample, na.rm = T),
    `Standard deviation` = sd(ddPCR_reads_per_sample, na.rm = T),
    `Interquartile range` = IQR(ddPCR_reads_per_sample, na.rm = T)) %>%
  mutate_if(is.double, ~round(., digits = 2)) %>%
  pivot_longer(cols = -sample, names_to = "metric", values_to = "value") %>%
  pivot_wider(names_from = sample, values_from = value) %>%
  dplyr::select(metric, IsoCtrl, Swab)
```

```
## # A tibble: 6 x 3
##   metric      IsoCtrl      Swab
##   <chr>      <dbl>      <dbl>
## 1 Mean      34.2      42751319.
## 2 Median     8.15      469164.
## 3 Minimum    2.67         0
## 4 Maximum   274.     807827799.
## 5 Standard deviation 77.0 105362119.
## 6 Interquartile range 9.2 23326779.
```

```
# visualization
all_data %>%
  mutate(sample = ifelse(sample == "IsoCtrl", "Isolation Control", "Swab Specimen")) %>%
  ggplot(aes(x = sample, y = ddPCR_reads_per_sample, color = sample)) +
  geom_boxplot() +
  geom_point(size = 1, position = position_jitter(width = 0.25)) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        legend.position = "none",
        text = element_text(family = "sans"),
        axis.title.y = element_text(size = 12),
        axis.text.y = element_text(size = 10),
        axis.text.x = element_text(size = 10)) +
  labs(x = NULL, y = expression(log[10] ~ " Mean ddPCR reads per sample")) +
  scale_y_log10(labels = scales::trans_format("log10", math_format(10^.x))) +
  scale_color_manual(values = c("#D55E00", "#cc79a7"))
```



B. Cases (developed AKI) versus Controls (did not develop AKI)

```
# bacterial density table by AKI
all_data_swabs %>%
  group_by(aki) %>%
  summarize(`Mean` = mean(ddPCR_reads_per_sample, na.rm = T),
```

```

`Median` = median(ddPCR_reads_per_sample,na.rm = T),
`Minimum` = min(ddPCR_reads_per_sample,na.rm = T),
`Maximum` = max(ddPCR_reads_per_sample,na.rm = T),
`Standard deviation` = sd(ddPCR_reads_per_sample,na.rm = T),
`Interquartile range` = IQR(ddPCR_reads_per_sample,na.rm = T)) %>%
mutate_if(is.double,~round(., digits = 2)) %>%
pivot_longer(cols = -aki, names_to = "metric", values_to = "value") %>%
pivot_wider(names_from = aki, values_from = value)

```

```

## # A tibble: 6 x 3
##   metric          '0'          '1'
##   <chr>          <dbl>        <dbl>
## 1 Mean          38534094.   51185768.
## 2 Median        190385.    1785092.
## 3 Minimum         0         610.
## 4 Maximum       807827799.  678881226.
## 5 Standard deviation 98592385. 117713691.
## 6 Interquartile range 11835683. 34062600.

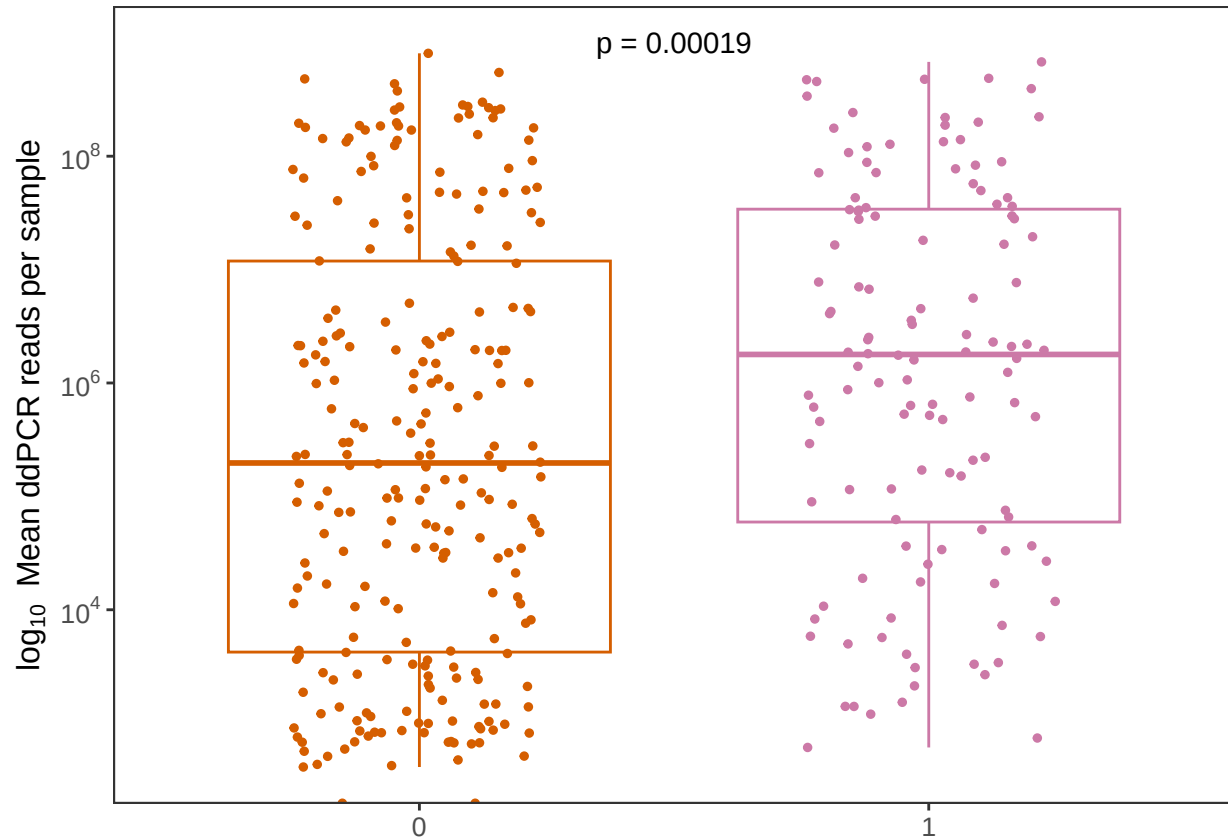
```

```

# make zero ddPCR reads very small number for log transformation
all_data_ddPCR <- all_data_swabs %>%
  mutate(ddPCR_reads_per_sample = ifelse(ddPCR_reads_per_sample == 0, 0.00001, ddPCR_reads_per_sample))
# statistical testing (unpaired t-test)
density_aki_test <- t.test(log(ddPCR_reads_per_sample) ~ aki, data = all_data_ddPCR)
density_aki_pvalue <- density_aki_test$p.value

# visualization
all_data_swabs %>%
  mutate(sample = ifelse(aki == 0, "No AKI", "AKI")) %>%
  ggplot(aes(x = aki, y = ddPCR_reads_per_sample, color = aki)) +
  geom_boxplot() +
  geom_point(size = 1, position = position_jitter(width = 0.25)) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        legend.position = "none",
        text = element_text(family = "sans"),
        axis.title.y = element_text(size = 12),
        axis.text.y = element_text(size = 10),
        axis.text.x = element_text(size = 10)) +
  labs(x = NULL, y = expression(log[10] ~ " Mean ddPCR reads per sample")) +
  scale_y_log10(labels = scales::trans_format("log10", math_format(10^.x))) +
  scale_color_manual(values = c("#D55E00", "#cc79a7")) +
  annotate("text", x = 1.5, y = 1e09,
         label = paste("p =", formatC(density_aki_pvalue, digits = 2)),
         size = 4, family = "sans")

```



III. Community Composition

A. Principal Component Analysis

1. Sequencing Controls versus Swab Specimens

```
otu.hel <- all_data %>%
  dplyr::select(contains("Otu")) %>%
  decostand("hellinger") # standardize OTU columns
tidy_pca <- prcomp(otu.hel)

# create data frame to plot PC1 and PC2
plot_by_sample <- data.frame(tidy_pca$x) %>%
  mutate(sample_name = all_data$sample_name) %>%
  dplyr::select(sample_name, PC1, PC2) %>%
  mutate(size = sqrt(PC1^2 + PC2^2)) %>%
  inner_join(all_data %>%
    dplyr::select(sample_name, sample)) %>%
  mutate(sample = case_when(sample == "IsoCtrl" ~ "Isolation Control",
    sample == "Swab" ~ "Swab Specimen"))

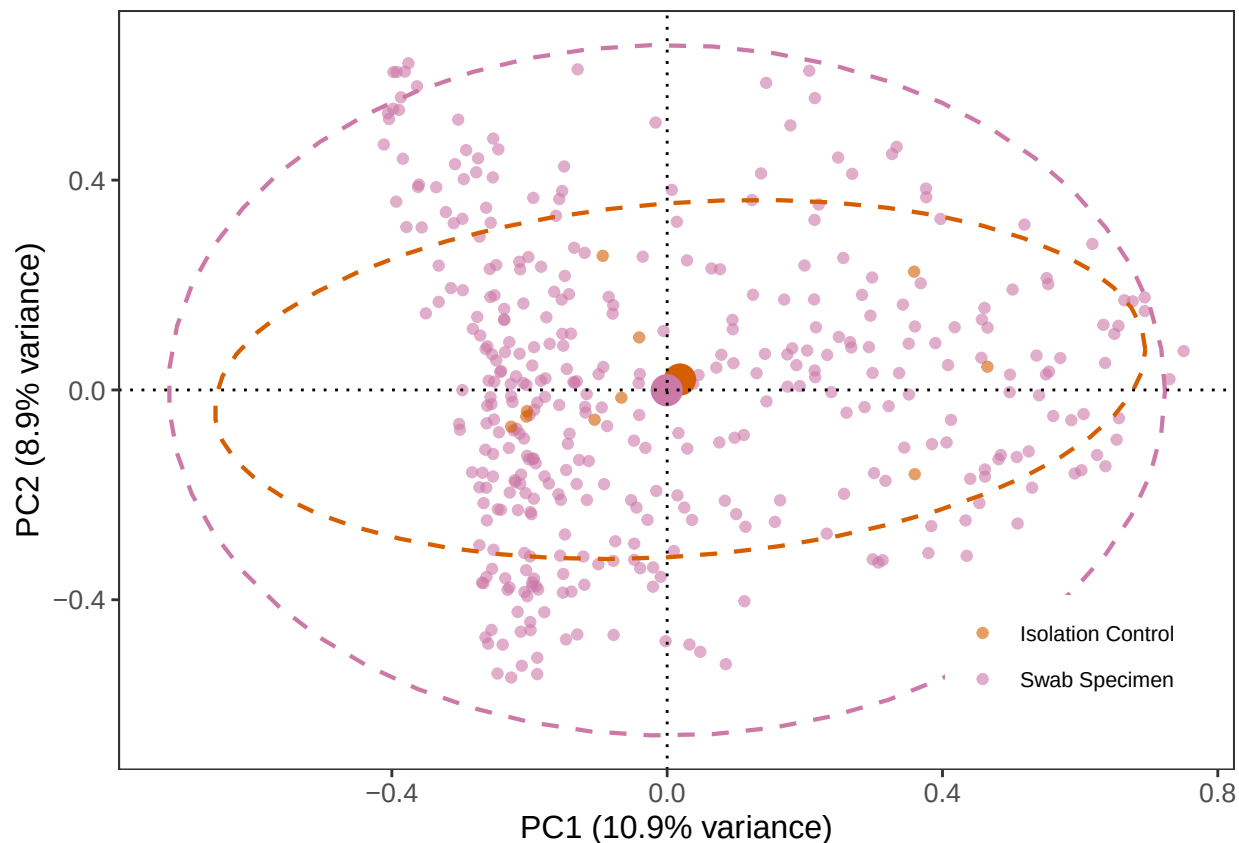
# get centroids for each sample type
find_centroids <- plot_by_sample %>%
  group_by(sample) %>%
```

```

    summarize(PC1_mean = mean(PC1),
              PC2_mean = mean(PC2))
# percent of PC explained
pca_var <- tidy_pca$sdev^2
pca_var_perc <- round(pca_var / sum(pca_var) * 100, 1)
pc1_perc <- pca_var_perc[1]
pc2_perc <- pca_var_perc[2]

# plot principal component analysis
ggplot(data = plot_by_sample) +
  geom_point(aes(x = PC1, y = PC2, color = sample), alpha = 0.6) +
  geom_point(data = find_centroids, aes(x = PC1_mean, y = PC2_mean, color = sample),
            size = 5, show.legend = FALSE) +
  stat_ellipse(aes(x = PC1, y = PC2, color = sample),
              type = "norm", linetype = 2, size = 0.75, show.legend = FALSE) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        legend.title = element_blank(),
        axis.title = element_text(size = 12, family = "sans"),
        axis.text = element_text(size = 10, family = "sans"),
        legend.text = element_text(size = 8, family = "sans"),
        legend.position = c(0.85, 0.15)) +
  scale_color_manual(values = c(c("#D55E00", "#CC79A7")
)) +
  geom_vline(xintercept = 0, linetype = 3) +
  geom_hline(yintercept = 0, linetype = 3) +
  labs(x = paste0("PC1 (", pc1_perc, "% variance)"),
       y = paste0("PC2 (", pc2_perc, "% variance)"))

```



2. Cases (developed AKI) versus Controls (did not develop AKI)

```
otu.hel.swabs <- all_data_swabs %>%
  dplyr::select(contains("Otu")) %>%
  decostand("hellinger")
tidy_swab_pca <- prcomp(otu.hel.swabs)

# create data frame to plot PC1 and PC2
plot_by_aki <- data.frame(tidy_swab_pca$x) %>%
  mutate(sample_name = all_data_swabs$sample_name) %>%
  dplyr::select(sample_name, PC1, PC2) %>%
  mutate(size = sqrt(PC1^2 + PC2^2)) %>%
  inner_join(all_data_swabs %>%
    dplyr::select(sample_name, aki)) %>%
  mutate(aki = case_when(aki == 1 ~ "AKI",
    aki == 0 ~ "No AKI"))

# get centroids for each sample type
find_centroids <- plot_by_aki %>%
  group_by(aki) %>%
  summarize(PC1_mean = mean(PC1),
    PC2_mean = mean(PC2))

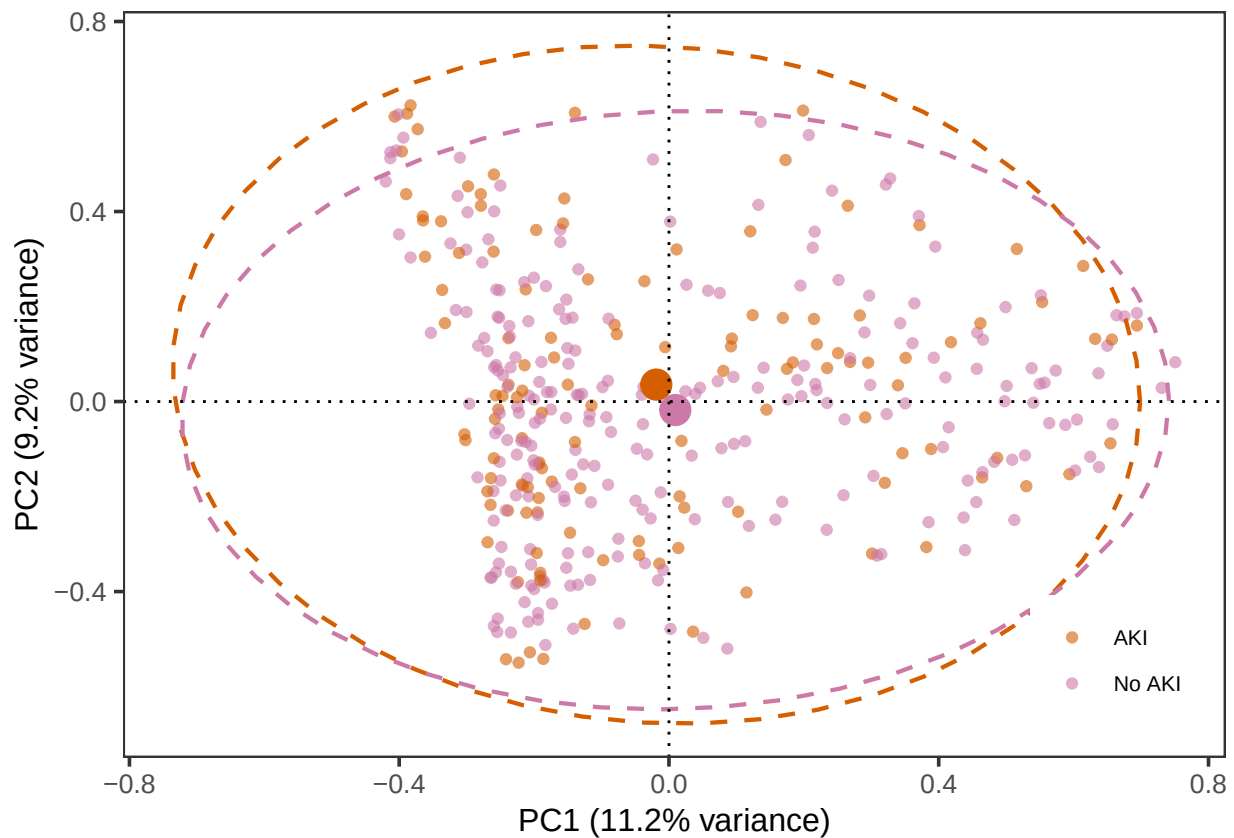
# percent of PC explained
pca_var <- tidy_swab_pca$sdev^2
pca_var_perc <- round(pca_var / sum(pca_var) * 100, 1)
```

```

pc1_perc <- pca_var_perc[1]
pc2_perc <- pca_var_perc[2]

# plot principal component analysis
ggplot(data = plot_by_aki) +
  geom_point(aes(x = PC1, y = PC2, color = aki), alpha = 0.6) +
  geom_point(data = find_centroids,
             aes(x = PC1_mean, y = PC2_mean, color = aki),
             size = 5, show.legend = FALSE) +
  stat_ellipse(aes(x = PC1, y = PC2, color = aki),
              type = "norm", linetype = 2, size = 0.75, show.legend = FALSE) +
  theme_bw(base_size = 14) +
  theme(panel.grid = element_blank(),
        legend.title = element_blank(),
        axis.title = element_text(size = 12, family = "sans"),
        axis.text = element_text(size = 10, family = "sans"),
        legend.text = element_text(size = 8, family = "sans"),
        legend.position = c(0.9, 0.13)) +
  scale_color_manual(values = c(c("#D55E00", "#CC79A7")
)) +
  geom_vline(xintercept = 0, linetype = 3) +
  geom_hline(yintercept = 0, linetype = 3) +
  labs(x = paste0("PC1 (", pc1_perc, "% variance)"),
       y = paste0("PC2 (", pc2_perc, "% variance)"))

```



B. Hypothesis Testing

1. PERMANOVA

```
set.seed(2305)
# sequencing controls versus swab specimens
adonis2(otu.hel ~ all_data$sample, method = "euclidean", permutations = 10000)

## Permutation test for adonis under reduced model
## Terms added sequentially (first to last)
## Permutation: free
## Number of permutations: 10000
##
## adonis2(formula = otu.hel ~ all_data$sample, permutations = 10000, method = "euclidean")
##              Df SumOfSqs      R2      F Pr(>F)
## all_data$sample  1      1.711 0.0057 2.1884 0.0013 **
## Residual        382    298.640 0.9943
## Total           383    300.350 1.0000
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# cases (developed AKI) versus controls (did not develop AKI)
# stratified by matched pair
adonis2(otu.hel.swabs ~ all_data_swabs$aki, strata = all_data_swabs$match_id,
        by = "terms", permutations = 10000)
```

```
## Permutation test for adonis under reduced model
## Terms added sequentially (first to last)
## Blocks: strata
## Permutation: free
## Number of permutations: 10000
##
## adonis2(formula = otu.hel.swabs ~ all_data_swabs$aki, permutations = 10000, by = "terms", strata = a
##              Df SumOfSqs      R2      F Pr(>F)
## all_data_swabs$aki  1      0.805 0.00603 2.2458 0.0013 **
## Residual          370    132.677 0.99397
## Total             371    133.483 1.00000
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

2. mvabund

```
# a <- Sys.time() # keep track of the time with start/stop points
# otu_mva <- mvabund(all_data_swabs[,str_which(colnames(all_data_swabs), "Otu")]) # select only Otu col
# otu_ven_many <- manyglm(otu_mva ~ all_data_swabs$aki)
# otu_mva_results <- anova(otu_ven_many, p.uni="adjusted")
# b <- Sys.time()
# b - a
#
# plot(otu_ven_many) # Residual plot
```

```

# otu_mva_results$table # Signif
#
# # restructure, sort, filter to retrieve significant OTUs
# otu_mva_results_pvals <- data.frame(OTU = colnames(otu_mva_results$uni.p), uni_p = otu_mva_results$un
#   filter(uni_p <= 0.05) %>% # pval threshold
#   droplevels() %>%
#   arrange(uni_p) # order pvals
# # label OTUs
# otu_mva_results_pvals$OTU <- fct_inorder(paste0(otu_good_taxonomy[otu_mva_results_pvals$OTU, ]$Genus,
# otu_mva_results_pvals

```

C. Random Forest

```

# will use SMOTE (synthetic minority oversampling technique) because of unbalanced matching
# creates a new dataset by oversampling observations from minority class (i.e., cases)
# produces a dataset that has more balanced classes

# prep the data
clean_data_aki <- all_data_swabs %>%
  mutate(aki = as.factor(aki)) %>%
  dplyr::select(-nearZeroVar(.), -sample_name:-run, aki, match_id)

set.seed(4763)
# create a custom index for cross-validation with respect to matched pairs
matched_folds <- groupKFold(clean_data_aki$match_id, k = 5)
# define train control
myControl <- trainControl(
  method = "cv",
  number = 10,
  index = matched_folds,
  verboseIter = FALSE,
  sampling = "smote" # handle class imbalance
)
# define the extended parameter grid for tuning
gridsearch_aki <- expand.grid(
  mtry = seq(1, 20, by = 1), # increase range of mtry
  splitrule = c("gini", "extratrees"), # explore multiple split rules
  min.node.size = c(1, 5, 10) # try different minimum node sizes
)

# train the random forest model
suppressMessages({
  otu_aki_model <- train(
    aki ~ . -match_id,
    method = "ranger",
    importance = "permutation",
    oob.error = TRUE,
    seed = 4763,
    trControl = myControl,
    tuneGrid = gridsearch_aki,
    data = clean_data_aki
  )

```

```

    )
  })
  # output the final selected parameters
  print(otu_aki_model$bestTune)

##      mtry splitrule min.node.size
## 103    18      gini              1

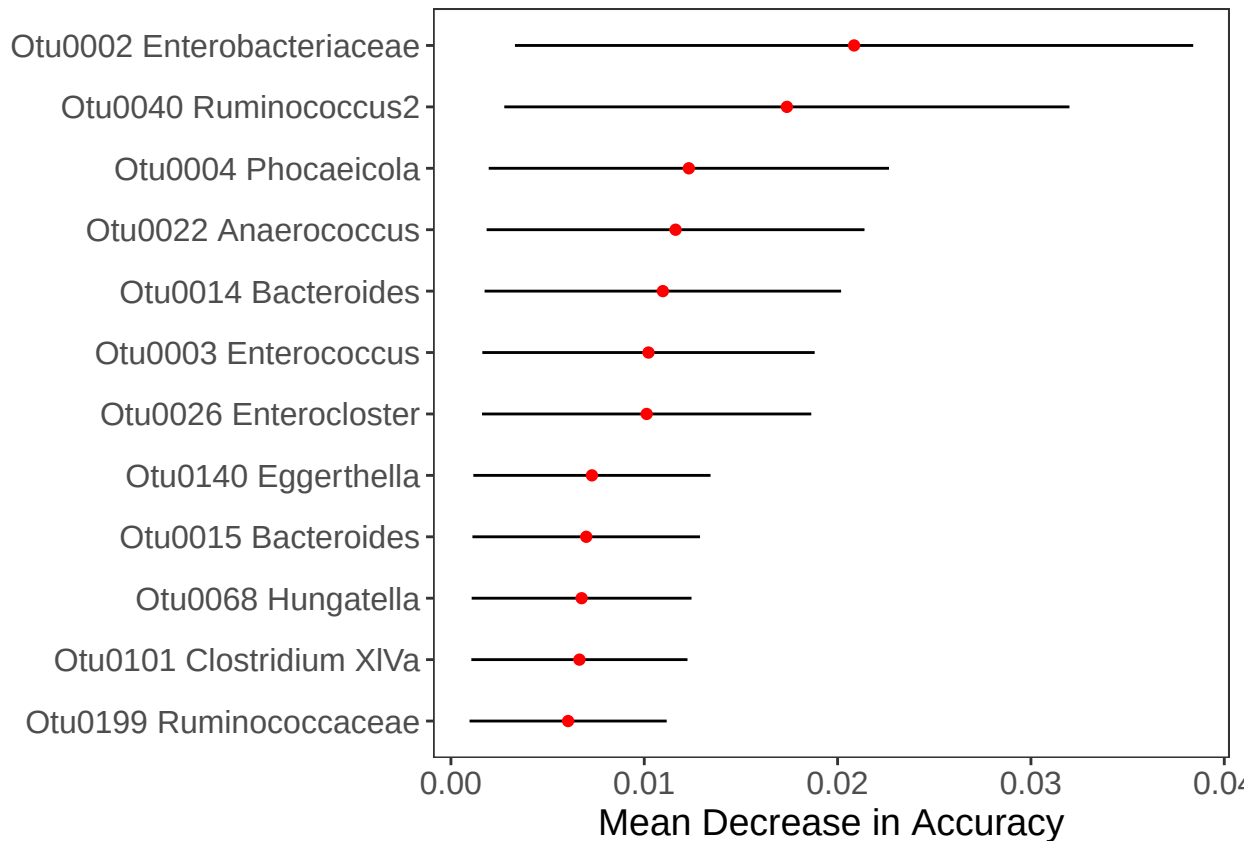
set.seed(4763)
important_features_aki <- ranger::importance_pvalues(
  otu_aki_model$finalModel,
  method = "altmann",
  formula = aki ~ . -match_id,
  data = clean_data_aki) %>%
  as.data.frame() %>%
  rownames_to_column() %>%
  rename(features = "rowname")

otus_for_aki <- important_features_aki %>%
  filter(importance > 0,
    pvalue < 0.01) %>%
  inner_join(otu_good_taxonomy, by = c("features" = "OTU")) %>%
  mutate(name = str_c(features, Genus, sep = " ")) %>%
  rename(`Mean Decrease in Accuracy` = "importance") %>%
  mutate(ci = `Mean Decrease in Accuracy`/qnorm(pvalue, lower.tail = FALSE),
    lower_ci = `Mean Decrease in Accuracy` -1.96*ci,
    upper_ci = `Mean Decrease in Accuracy` +1.96*ci,
    name = str_remove(name, "_unclassified"),
    name = str_remove(name, " incertae sedis"),
    name = str_replace(name, "_", " "),
    name = factor(name),
    name = fct_reorder(name, `Mean Decrease in Accuracy`)) %>%
  arrange(desc(lower_ci))

important_otus <- otus_for_aki %>%
  arrange(desc(`Mean Decrease in Accuracy`)) %>%
  head(12) %>%
  pull(features)

feature_importance_plot <- otus_for_aki %>%
  head(12) %>%
  ggplot(aes(x = name, y = `Mean Decrease in Accuracy`)) +
  geom_segment(aes(x = name, xend = name, y = lower_ci, yend = upper_ci)) +
  geom_point(color = "red") +
  coord_flip() +
  theme_bw() +
  theme(panel.grid = element_blank(),
    text = element_text(family = "sans"),
    axis.text.y = element_text(size = 12, family = "sans"),
    axis.title.x = element_text(size = 14, family = "sans"),
    axis.text.x = element_text(size = 12, family = "sans")) +
  labs(x = NULL, y = "Mean Decrease in Accuracy")
feature_importance_plot

```

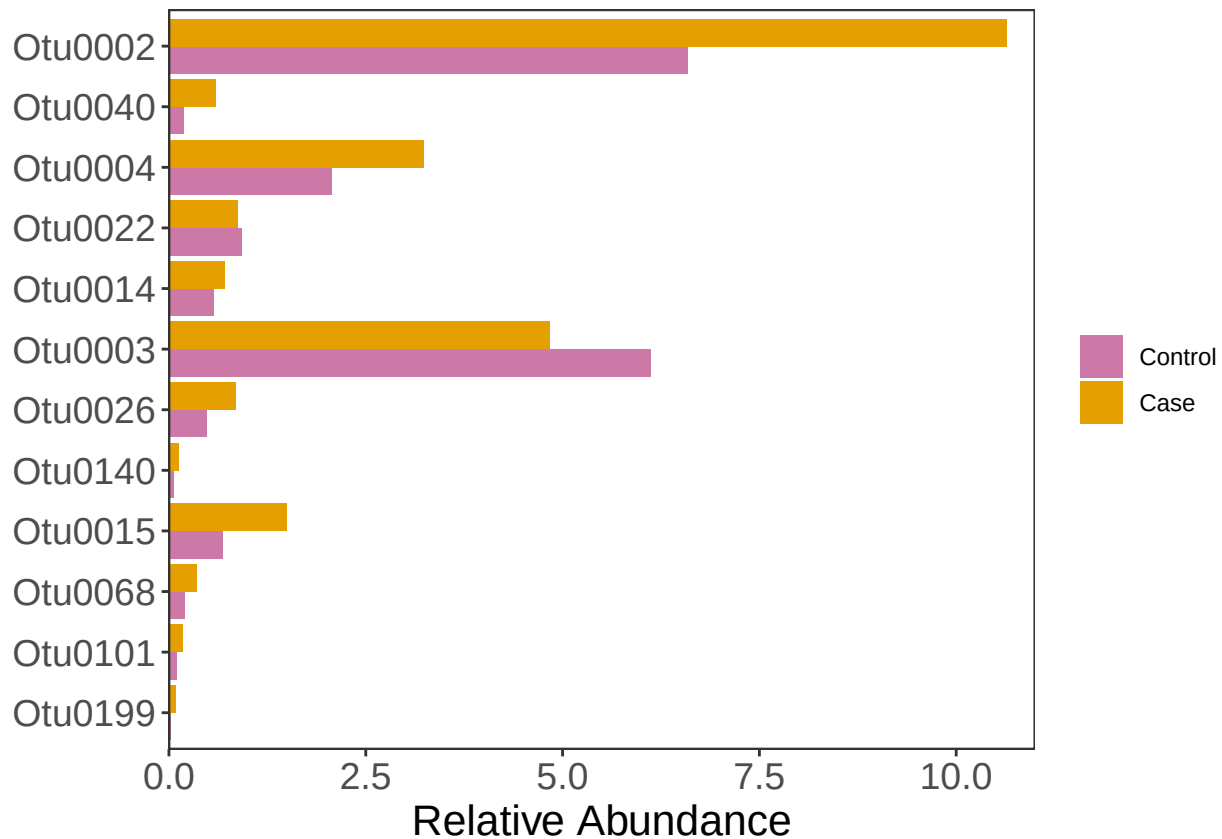


1. Relative Abundance of Random Forest-identified OTUs

```
# get mean relative abundance for RF-identified OTUs
rf_otus <- all_data_swabs %>%
  dplyr::select(aki, important_otus) %>%
  pivot_longer(col = c(-aki), names_to = "OTU", values_to = "rel_abund") %>%
  group_by(OTU, aki) %>%
  mutate(mean_ra = mean(rel_abund)) %>%
  select(-rel_abund) %>%
  group_by(aki) %>%
  distinct(OTU, .keep_all = T)

# plot relative abundance
ggplot(rf_otus, aes(x = mean_ra, y = OTU, fill = aki)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  theme_minimal() +
  scale_y_discrete(limits = rev(important_otus)) +
  scale_x_continuous(limits = c(0, 11), expand = c(0,0)) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        legend.title = element_blank(),
        axis.title.x = element_text(size = 16, family = "sans"),
        axis.title.y = element_text(size = 16),
        axis.text = element_text(size = 14)) +
  labs(y = NULL,
```

```
x = "Relative Abundance") +
scale_fill_manual(values = c("#cc79a7", "#E69F00"), labels = c("Control", "Case"))
```



IV. Community Diversity

A. Beta-diversity: across specimens

```
# Use helper function will to disambiguate columns associated with paired BC comparisons
bc_rename <- function(colname) {
  if(str_detect(colname, ".x$")) { # For .x, simply remove the suffix
    return(str_remove(colname, ".x$"))
  } else if (str_detect(colname, ".y$")) {
    return(str_replace(colname, ".y$", "_comparison")) # For .y, replace with _comparison suffix
  } else { # Otherwise name does not need adjustment
    return(colname)
  }
}

bc_df <- all_data_swabs %>%
  select(contains("Otu")) %>% # keep only OTU columns
  vegdist(diag = TRUE, upper = TRUE) %>% # method = "bray" by default
  as.matrix() %>%
```

```

as.data.frame(row.names = all_data_swabs$sample_name) %>%
# add column names
setNames(all_data_swabs$sample_name) %>%
# create specimen column from rownames
rownames_to_column("sample_name") %>%
# transform from wide to long
pivot_longer(cols = -sample_name, names_to = "comparison", values_to = "BC") %>%
# remove comparisons against the same specimen (the diagonal in the original distance matrix)
filter(sample_name != comparison) %>%
# join cols for subsetting by specimen1
left_join(select(all_data_swabs, sample_name:aki), by = "sample_name") %>%
# join cols for specimen2 (comparison)
left_join(select(all_data_swabs, sample_name:aki),
          by = c("comparison" = "sample_name")) %>%
# use the custom renaming function defined above
rename_with(~ map_chr(., bc_rename))

# subset to retain only the case vs control comparisons
bc_aki_df <- bc_df %>%
  filter(aki == aki_comparison) %>%
  droplevels() %>%
  group_by(sample_name, aki) %>%
  summarize(Mean_BC = mean(BC),
            SEM_BC = sqrt(var(BC)/length(BC)), .groups = "drop")

```

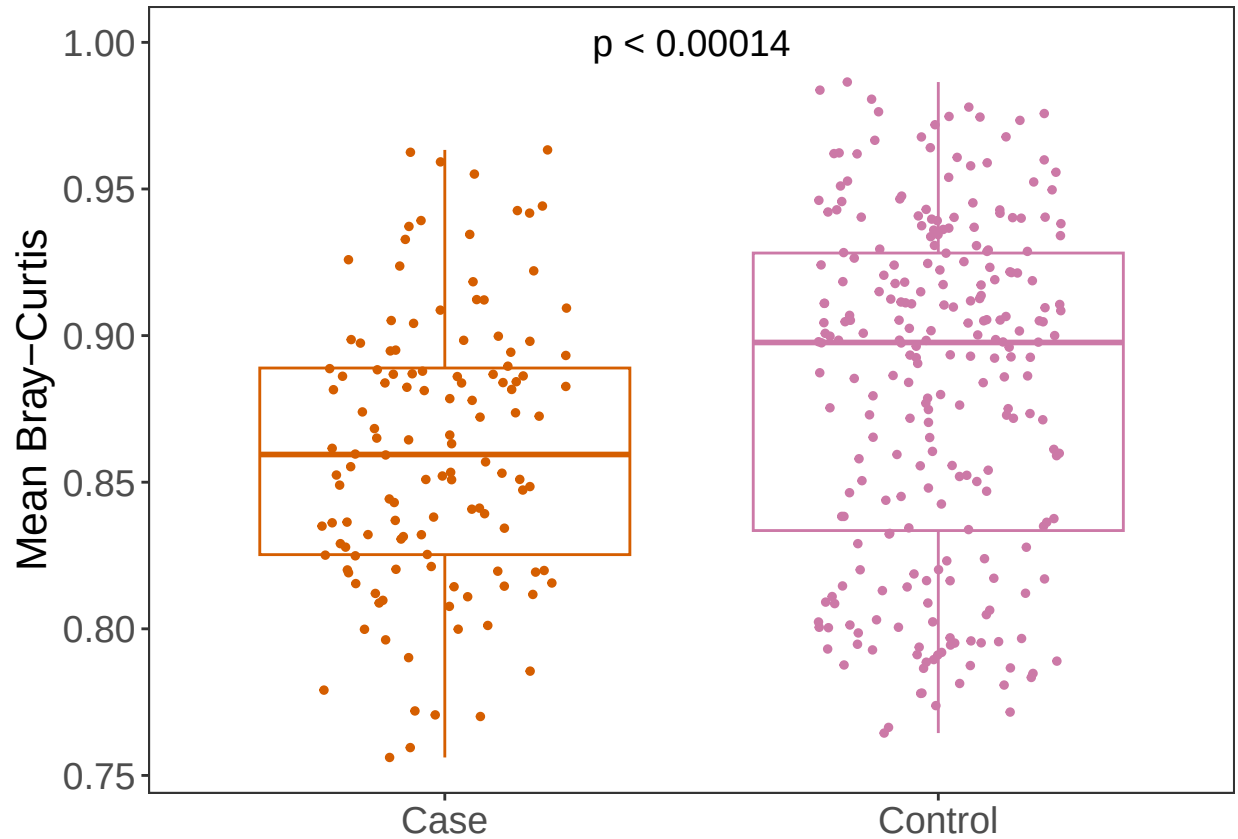
Visualization

```

# statistical testing (unpaired t-test)
beta_div_test <- t.test(Mean_BC ~ aki, data = bc_aki_df)
beta_div_pvalue <- beta_div_test$p.value

beta_vis <- bc_aki_df %>%
  mutate(aki = ifelse(aki == 1, "Case", "Control")) %>%
  ggplot(aes(x = aki, y = Mean_BC, color = aki)) +
  geom_boxplot() +
  geom_point(size = 1, position = position_jitter(width = 0.25)) +
  theme_bw() +
  labs(y = "Mean Bray-Curtis", x = NULL) +
  theme(panel.grid = element_blank(),
        legend.position = "none",
        axis.title.x = element_text(size = 16, family = "sans"),
        axis.title.y = element_text(size = 16, family = "sans"),
        axis.text = element_text(size = 14, family = "sans")
  ) +
  scale_color_manual(values = c("#D55E00", "#cc79a7")) +
  annotate("text", x = 1.5, y = 1,
          label = paste("p <", formatC(beta_div_pvalue, digits = 2)),
          size = 5, family = "sans")
beta_vis

```



B. Alpha-diversity: within specimens

```
# create data frame with sample_name, Shannon, Total Reads, Dominance on a per sample basis
alpha_div_df <- bind_cols(sample_name = all_data_swabs$sample_name,
  Shannon = diversity(all_data_swabs %>%
    dplyr::select(contains("Otu"))),
  # Shannon diversity per specimen
  Total_Reads = pmap_dbl(data.frame(all_data_swabs %>%
    dplyr::select(contains("Otu"))), sum),
  # rowwise sum (by specimen)
  Dominance = pmap_dbl(dplyr::select(all_data_swabs,
    contains("Otu")), max)) %>%
  # Dominance: max % of top OTU per specimen
  left_join(all_data_swabs[,1:6], by = "sample_name") # join metadata columns
```

Visualization

```
# statistical testing (unpaired t-test)
alpha_div_test <- t.test(Shannon ~ aki, data = alpha_div_df)
alpha_div_pvalue <- alpha_div_test$p.value

shannon <- alpha_div_df %>%
  mutate(aki = ifelse(aki == 1, "Case", "Control")) %>%
```

```

ggplot(aes(x = aki, y = Shannon, color = aki)) +
  geom_boxplot() +
  geom_point(size = 1, position = position_jitter(width = 0.25)) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        legend.position = "none",
        axis.title.x = element_text(size = 16, family = "sans"),
        axis.title.y = element_text(size = 16, family = "sans"),
        axis.text = element_text(size = 14, family = "sans")
  ) +
  labs(x = NULL, y = "Shannon") +
  scale_color_manual(values = c("#D55E00", "#cc79a7")) +
  annotate("text", x = 1.5, y = 4,
         label = paste("p =", formatC(alpha_div_pvalue, digits = 2)), size = 5)
shannon

```

